

Algorithm 1: FIFO PAGE REPLACEMENT ALGORITHM

Objective

The objective of this experiment is to study and implement the FIFO (First In First Out) page replacement algorithm and observe its behavior by calculating the number of page faults for a given page reference string and frame capacity.

Theory

FIFO is one of the simplest page replacement algorithms. In this algorithm, the page that enters the memory first is the first one to be removed when a page fault occurs and the memory is full. The algorithm does not consider how frequently or recently a page is used. It strictly follows the order of arrival of pages into memory.

Algorithm

The algorithm maintains a queue to store the order of pages loaded into memory and a set to quickly check whether a page is already present. When a page fault occurs and the memory is full, the page at the front of the queue is removed.

Code

```
#include <bits/stdc++.h>

using namespace std;

int pagefaults(int n,vector<int> pages, int capacity)
{
    unordered_set<int> s;
    queue<int> q;
    int pf = 0;
    for (int page : pages)
    {
        if (s.find(page) == s.end())
        {
            pf++;
            if (q.size() == capacity)
            {
                int old = q.front();
                q.pop();
                s.erase(old);
            }
            q.push(page);
        }
    }
}
```

```

        s.insert(page);
    }
}
return pf;
}
void marwan()
{
    int n;
    cin >> n;
    vector<int> v(n);
    for (auto &vl : v)
        cin >> vl;
    int capacity;
    cin >> capacity;
    cout << pagefaults(n, v, capacity) << endl;
}
int32_t main()
{
    ios_base::sync_with_stdio(false);
    marwan();
    return 0;
}

```

Input

Number of pages:

7

Page reference string:

1 2 3 1 4 5 1

Frame capacity:

3

Output

6

Step by Step Execution

Page 1 loaded, page fault occurs. Frames: 1
Page 2 loaded, page fault occurs. Frames: 1 2
Page 3 loaded, page fault occurs. Frames: 1 2 3
Page 1 accessed, already present. No page fault. Frames: 1 2 3
Page 4 loaded, page fault occurs. Page 1 is removed. Frames: 2 3 4
Page 5 loaded, page fault occurs. Page 2 is removed. Frames: 3 4 5
Page 1 loaded, page fault occurs. Page 3 is removed. Frames: 4 5 1

Result

Total number of page faults using FIFO algorithm is 6.

Conclusion

FIFO is easy to implement but does not consider the importance of frequently used pages. This can lead to unnecessary page faults, especially in cases where old pages are still actively used.

Algorithm 2: LRU PAGE REPLACEMENT ALGORITHM

Objective

The objective of this experiment is to implement the LRU (Least Recently Used) page replacement algorithm and analyze its performance in terms of page faults using the same input data.

Theory

LRU replaces the page that has not been used for the longest period of time. It assumes that pages used recently will likely be used again soon. This algorithm generally performs better than FIFO because it uses past access behavior to make replacement decisions.

Algorithm

The algorithm uses a list to maintain the order of page usage, where the front represents the least recently used page and the back represents the most recently used page. A map is used to track the position of each page in the list. On a page hit, the page is moved to the back of the list. On a page fault with full memory, the page at the front is removed.

Code

```
#include <bits/stdc++.h>

using namespace std;

int pagefaults(int n, vector<int> pages, int capacity)
{
    list<int> l;
    unordered_map<int, list<int>::iterator> mp;
    int pf = 0;
```

```

for (int page : pages)
{
    if (mp.find(page) == mp.end())
    {
        pf++;
        if (l.size() == capacity)
        {
            int lru = l.front();
            l.pop_front();
            mp.erase(lru);
        }
    }
    else
    {
        l.erase(mp[page]);
    }

    l.push_back(page);
    mp[page] = --l.end();
}
return pf;
}

void marwan()
{
    int n;
    cin >> n;
    vector<int> v(n);
    for (auto &vl : v)
        cin >> vl;
    int capacity;

```

```

    cin >> capacity;

    cout << pagefaults(n, v, capacity) << endl;
}

int32_t main()
{
    ios_base::sync_with_stdio(false);
    marwan();
    return 0;
}

```

Input

Number of pages:

7

Page reference string:

1 2 3 1 4 5 1

Frame capacity:

3

Output

5

Step by Step Execution

Page 1 loaded, page fault occurs. Frames: 1

Page 2 loaded, page fault occurs. Frames: 1 2

Page 3 loaded, page fault occurs. Frames: 1 2 3

Page 1 accessed, page hit. Frames updated to: 2 3 1

Page 4 loaded, page fault occurs. Page 2 removed. Frames: 3 1 4

Page 5 loaded, page fault occurs. Page 3 removed. Frames: 1 4 5

Page 1 accessed, page hit. Frames updated to: 4 5 1

Result

Total number of page faults using LRU algorithm is 5.

Conclusion

LRU performs better than FIFO for most of the inputs because it keeps track of recent usage and avoids removing pages that are still frequently accessed. This results in fewer page faults and better memory utilization.